htmlParser.ts

```
export interface ParsedQuestion {
  id: number;
  text: string;
  options: {
    id: string;
    text: string;
    isCorrect: boolean;
  }[];
  solution?: string;
  images?: string[];
  solutionImages?: string[];
}

export interface ParsedQuestions {
  questions: ParsedQuestion[];
  title?: string;
  totalQuestions?: number;
  extractedImages?: number;
}

export function parseHTMLContent(htmlContent: string): ParsedQuestions {
  // Create a temporary DOM element to parse the HTML
  const parser = new DOMParser();
  const doc = parser.parseFromString(htmlContent, 'text/html');

  const questions: ParsedQuestion[] = [];
  let title = '';
  let extractedImages = 0;

  // Extract title from TakeExamHeader
  const titleElement = doc.querySelector('.TakeExamHeader h3:last-child');
  if (titleElement) {
    title = titleElement.textContent?.trim() || '';
  }

  // Find all question blocks
  const questionBlocks = doc.querySelectorAll('.questionBlock');

  questionBlocks.forEach((block, index) => {
    const questionId = index + 1;
```

```
// Extract question text
const questionTextElement = block.querySelector('.questionText');
let questionText = '';

if (questionTextElement) {
  // Clean up the question text and preserve mathematical content
  questionText = cleanTextContent(questionTextElement);
}

// Extract images from question section only (not from solution)
const questionImages = extractImagesFromSection(block, '.questionText');
if (questionImages.length > 0) {
  extractedImages += questionImages.length;
}

// Extract options
const optionElements = block.querySelectorAll('.questionOption');
const options: { id: string; text: string; isCorrect: boolean }[] = [];

optionElements.forEach(optionElement => {
  const optionId = optionElement.querySelector('.input-group-text')?.textContent?.trim() || '';
  const optionLabel = optionElement.querySelector('.questionTable label');
  const isCorrect = optionElement.querySelector('.fas.fa-check') !== null;

  let optionText = '';
  if (optionLabel) {
    optionText = cleanTextContent(optionLabel);
  }

  if (optionId && optionText) {
    options.push({
      id: optionId,
      text: optionText,
      isCorrect
    });
  }
});

// Extract solution if exists
const solutionElement = block.querySelector('.solveText');
let solution = '';
let solutionImages: string[] = [];

if (solutionElement) {
```

```
      solution = cleanTextContent(solutionElement);
      // Extract images from solution section
      solutionImages = extractImagesFromSection(block, '.solveText');
      if (solutionImages.length > 0) {
        extractedImages += solutionImages.length;
      }
    }

    // Only add question if it has content
    if (questionText && options.length > 0) {
      questions.push({
        id: questionId,
        text: questionText,
        options,
        solution: solution || undefined,
        images: questionImages.length > 0 ? questionImages : undefined,
        solutionImages: solutionImages.length > 0 ? solutionImages : undefined
      });
    }
  });

  return {
    questions,
    title: title || undefined,
    totalQuestions: questions.length,
    extractedImages
  };
}

function extractImagesFromSection(block: Element, selector: string): string[] {
  const images: string[] = [];
  const section = block.querySelector(selector);

  if (!section) return images;

  // For question images, exclude any images that are inside solution sections
  if (selector === '.questionText') {
    // Get all images in the question text section
    const allImages = section.querySelectorAll('img');
    allImages.forEach((img) => {
      // Check if this image is inside a solution section
      const closestSolveText = img.closest('.solveText');
      const closestQuestionText = img.closest('.questionText');
```

```
      // Only include if it's in question text and NOT in solve text
      if (closestQuestionText && !closestSolveText) {
        const src = img.getAttribute('src');
        if (src) {
          images.push(processImageUrl(src));
        }
      }
    });
  } else {
    // For solution images, only get images directly in the solution section
    const imgElements = section.querySelectorAll('img');
    imgElements.forEach((img) => {
      const src = img.getAttribute('src');
      if (src) {
        images.push(processImageUrl(src));
      }
    });
  }

  return images;
}

function processImageUrl(src: string): string {
  // Convert relative URLs to absolute if needed
  if (src.startsWith('data:')) {
    // Data URLs are already complete
    return src;
  } else if (src.startsWith('http://') || src.startsWith('https://')) {
    // Absolute URLs
    return src;
  } else {
    // Relative URLs - try to construct absolute URL
    try {
      const baseUrl = window.location.origin;
      const absoluteUrl = new URL(src, baseUrl).href;
      return absoluteUrl;
    } catch {
      // If URL construction fails, use as-is
      return src;
    }
  }
}
```

```
function cleanTextContent(element: Element): string {
  // Clone the element to avoid modifying the original
  const cloned = element.cloneNode(true) as Element;

  // Remove style tags and their content
  const styleTags = cloned.querySelectorAll('style');
  styleTags.forEach(tag => tag.remove());

  // Convert math-tex spans to LaTeX format
  const mathElements = cloned.querySelectorAll('.math-tex');
  mathElements.forEach(mathEl => {
    const mathContent = mathEl.textContent || '';
    // Keep the mathematical content as is - MathJax will handle it
    mathEl.replaceWith(document.createTextNode(mathContent));
  });

  // Clean up extra whitespace and return
  return cloned.textContent?.replace(/\s+/g, ' ').trim() || '';
}


export function extractMathJaxScript(htmlContent: string): string | null {
  // Extract MathJax configuration from the HTML
  const mathJaxConfigRegex = /window\.MathJax\s*=\s*\{[\s\S]*?\};/g;
  const match = htmlContent.match(mathJaxConfigRegex);

  if (match) {
    return match[0];
  }

  // Return default MathJax configuration
  return `
    window.MathJax = {
      tex: {
        inlineMath: [['$', '$'], ['\\\\(', '\\\\)']],
        displayMath: [['$$', '$$'], ['\\\\[', '\\\\]']]
      },
      svg: {
        fontCache: 'global'
      }
    };
  `;
}
```

global.d.ts

```
declare global {
  interface Window {
    MathJax?: {
      typesetPromise(): Promise<void>;
      tex?: {
        inlineMath?: string[][];
        displayMath?: string[][];
      };
      svg?: {
        fontCache?: string;
      };
    };
  }
}

export {};
```

upload.tsx

```
import { useState } from "react";
import { useLocation } from "wouter";
import { Button } from "@/components/ui/button";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Upload, FileText, AlertCircle } from "lucide-react";
import { parseHTMLContent, type ParsedQuestions } from "@/utils/htmlParser";

export default function UploadPage() {
  const [, setLocation] = useLocation();
  const [isDragging, setIsDragging] = useState(false);
  const [isProcessing, setIsProcessing] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const handleFileUpload = async (file: File) => {
    if (!file.type.includes('html') && !file.name.endsWith('.html')) {
      setError('Please upload an HTML file');
      return;
```

```
  }

  setIsProcessing(true);
  setError(null);

  try {
    const content = await file.text();
    const parsedData: ParsedQuestions = parseHTMLContent(content);

    if (parsedData.questions.length === 0) {
      setError('No questions found in the uploaded file');
      setIsProcessing(false);
      return;
    }

    // Store parsed data in localStorage for the quiz page
    localStorage.setItem('parsedQuestions', JSON.stringify(parsedData));

    // Navigate to quiz page
    setLocation('/quiz');
  } catch (err) {
    setError('Error processing the file. Please check the file format.');
    console.error('File processing error:', err);
  } finally {
    setIsProcessing(false);
  }
};

const handleDrop = (e: React.DragEvent) => {
  e.preventDefault();
  setIsDragging(false);

  const files = Array.from(e.dataTransfer.files);
  if (files.length > 0) {
    handleFileUpload(files[0]);
  }
};

const handleFileSelect = (e: React.ChangeEvent<HTMLInputElement>) => {
  const files = e.target.files;
  if (files && files.length > 0) {
    handleFileUpload(files[0]);
  }
};
```

```
return (
  <div className="min-h-screen bg-white p-4">
    <div className="max-w-2xl mx-auto pt-20">
      <div className="text-center mb-8">
        <h1 className="text-3xl font-bold text-gray-900 mb-2 font-kalpurush">
          প্রশ্ন আপলোড করুন
        </h1>
        <p className="text-gray-600">
          Upload your HTML file containing questions to create an interactive quiz
        </p>
      </div>

      <Card className="w-full bg-white">
        <CardHeader>
          <CardTitle className="flex items-center gap-2">
            <FileText className="w-5 h-5" />
            Upload HTML File
          </CardTitle>
        </CardHeader>
        <CardContent>
          <div
            className={`border-2 border-dashed rounded-lg p-12 text-center transition-colors ${
              isDragging
                ? 'border-primary bg-primary/5'
                : 'border-gray-300 hover:border-gray-400'
            }`}
            onDrop={handleDrop}
            onDragOver={(e) => {
              e.preventDefault();
              setIsDragging(true);
            }}
            onDragLeave={() => setIsDragging(false)}
          >
            <Upload className="mx-auto h-12 w-12 text-gray-400 mb-4" />
            <h3 className="text-lg font-medium text-gray-900 mb-2">
              {isProcessing ? 'Processing...' : 'Drop your HTML file here'}
            </h3>
            <p className="text-sm text-gray-500 mb-4">
              or click to browse and select a file
            </p>
            <input
              type="file"
              accept=".html"
```

```
              onChange={handleFileSelect}
              className="hidden"
              id="file-upload"
              disabled={isProcessing}
            />
            <Button
              onClick={() => document.getElementById('file-upload')?.click()}
              disabled={isProcessing}
              data-testid="button-upload-file"
            >
              {isProcessing ? 'Processing...' : 'Choose File'}
            </Button>
          </div>

          {error && (
            <div className="mt-4 p-4 bg-red-50 border border-red-200 rounded-lg">
              <div className="flex items-center gap-2 text-red-800">
                <AlertCircle className="w-4 h-4" />
                <span className="font-medium">Error:</span>
              </div>
              <p className="text-red-700 mt-1">{error}</p>
            </div>
          )}

          <div className="mt-6 text-sm text-gray-600">
            <h4 className="font-medium mb-2">Supported formats:</h4>
            <ul className="list-disc list-inside space-y-1">
              <li>HTML files from Utkorsho Online platform</li>
              <li>Files containing question blocks with multiple choice options</li>
              <li>Mathematical content with MathJax support</li>
            </ul>
          </div>
        </CardContent>
      </Card>
    </div>
  </div>
 );
}
```

quiz.tsx

```
import { useState, useEffect } from "react";
import { useLocation } from "wouter";
import { Button } from "@/components/ui/button";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { CheckCircle, XCircle, ArrowLeft, Lightbulb } from "lucide-react";
import { type ParsedQuestions, type ParsedQuestion } from "@/utils/htmlParser";

export default function QuizPage() {
  const [, setLocation] = useLocation();
  const [quizData, setQuizData] = useState<ParsedQuestions | null>(null);
  const [selectedAnswers, setSelectedAnswers] = useState<Record<number, string>>({});
  const [firstClickAnswers, setFirstClickAnswers] = useState<Record<number, string>>({});
  const [showSolutions, setShowSolutions] = useState<Record<number, boolean>>({});

  useEffect(() => {
    // Load parsed questions from localStorage
    const storedData = localStorage.getItem('parsedQuestions');
    if (storedData) {
      try {
        const parsed = JSON.parse(storedData) as ParsedQuestions;
        setQuizData(parsed);

        // Initialize MathJax if available
        if (window.MathJax) {
          window.MathJax.typesetPromise().then(() => {
            console.log('Mathematical equations rendered successfully');
          }).catch((err: any) => console.log('MathJax rendering error:', err));
        }
      } catch (err) {
        console.error('Error loading quiz data:', err);
        setLocation('/');
      }
    } else {
      setLocation('/');
    }
  }, [setLocation]);

  useEffect(() => {
    // Re-render MathJax when content changes
    if (window.MathJax && quizData) {
      window.MathJax.typesetPromise().catch((err: any) =>
        console.log('MathJax rendering error:', err)
```

```
    );
  }
}, [quizData, showSolutions]);

const handleOptionSelect = (questionId: number, optionId: string) => {
  setSelectedAnswers(prev => ({
    ...prev,
    [questionId]: optionId
  }));

  // Only record first click for accuracy calculation
  if (!firstClickAnswers[questionId]) {
    setFirstClickAnswers(prev => ({
      ...prev,
      [questionId]: optionId
    }));
  }
};

const toggleSolution = (questionId: number) => {
  setShowSolutions(prev => ({
    ...prev,
    [questionId]: !prev[questionId]
  }));
};

const isCorrectAnswer = (question: ParsedQuestion, optionId: string): boolean => {
  const option = question.options.find(opt => opt.id === optionId);
  return option?.isCorrect || false;
};

const getSelectedOption = (question: ParsedQuestion, selectedOptionId: string) => {
  return question.options.find(opt => opt.id === selectedOptionId);
};

const calculateAccuracy = (): number => {
  if (!quizData || Object.keys(firstClickAnswers).length === 0) return 0;

  let correctCount = 0;
  const totalAnswered = Object.keys(firstClickAnswers).length;

  Object.entries(firstClickAnswers).forEach(([questionId, optionId]) => {
    const question = quizData.questions.find(q => q.id === parseInt(questionId));
    if (question && isCorrectAnswer(question, optionId)) {
```

```
      correctCount++;
    }
  });

  return Math.round((correctCount / totalAnswered) * 100);
};

if (!quizData) {
  return (
    <div className="min-h-screen bg-white flex items-center justify-center">
      <div className="text-center">
        <h2 className="text-xl font-semibold text-gray-900 mb-2">Loading Quiz...</h2>
        <p className="text-gray-600">Please wait while we prepare your questions.</p>
      </div>
    </div>
  );
}


return (
  <div className="min-h-screen bg-white">
    {/* Header */}
    <div className="bg-white shadow-sm border-b border-gray-200 sticky top-0 z-10">
      <div className="max-w-4xl mx-auto px-4 py-4 flex items-center justify-between">
        <Button
          variant="ghost"
          onClick={() => setLocation('/')}
          className="flex items-center gap-2"
          data-testid="button-back-to-upload"
        >
          <ArrowLeft className="w-4 h-4" />
          Upload New File
        </Button>
        <div className="text-center">
          <h1 className="text-lg font-semibold text-gray-900 font-kalpurush">
            {quizData.title || 'Interactive Quiz'}
          </h1>
          <p className="text-sm text-gray-600">
            {quizData.questions.length} Questions • {calculateAccuracy()}% Accuracy
          </p>
        </div>
        <div className="w-[120px]"></div> {/* Spacer for centering */}
      </div>
    </div>
```

```
      {/* Questions */}
      <div className="max-w-4xl mx-auto px-4 py-6 space-y-6">
        {quizData.questions.map((question) => (
          <Card key={question.id} className="w-full bg-white">
            <CardHeader>
              <CardTitle className="flex items-center gap-3">
                <span className="bg-primary text-white px-3 py-1 rounded-full text-sm font-medium
font-kalpurush">
                  প্রশ্ন {question.id}
                </span>
              </CardTitle>
            </CardHeader>
            <CardContent className="space-y-4">
              {/* Question Text */}
              <div className="text-lg font-medium text-gray-800 font-kalpurush leading-relaxed">
                {question.text}
              </div>

              {/* Question Images */}
              {question.images && question.images.length > 0 && (
                <div className="flex flex-wrap gap-2 mb-3">
                  {question.images.map((imageUrl, index) => (
                    <div key={index} className="inline-block rounded overflow-hidden shadow-sm
bg-gray-50">
                      <img
                        src={imageUrl}
                        alt={`Question ${question.id} image ${index + 1}`}
                        className="max-w-xs max-h-40 object-contain"
                        onError={(e) => {
                          const target = e.target as HTMLImageElement;
                          target.style.display = 'none';
                        }}
                        data-testid={`img-question-${question.id}-${index}`}
                      />
                    </div>
                  ))}
                </div>
              )}

              {/* Options */}
              <div className="space-y-3">
                {question.options.map((option) => {
                  const isSelected = selectedAnswers[question.id] === option.id;
```

```
              const isCorrect = option.isCorrect;
              const showResult = isSelected;

              return (
                <div
                  key={option.id}
                  className={`flex items-center space-x-3 p-3 rounded-lg border cursor-pointer
transition-all ${
                    isSelected
                      ? isCorrect
                        ? 'border-green-500 bg-green-50'
                        : 'border-red-500 bg-red-50'
                      : 'border-gray-200 hover:border-primary hover:bg-blue-50'
                  }`}
                  onClick={() => handleOptionSelect(question.id, option.id)}
                  data-testid={`option-${question.id}-${option.id}`}
                >
                  <div className="flex-1 flex items-center justify-between">
                    <div className="flex items-center space-x-3">
                      <span className={`w-8 h-8 rounded-full flex items-center justify-center text-sm
font-medium ${
                        isSelected
                          ? isCorrect
                            ? 'bg-green-500 text-white'
                            : 'bg-red-500 text-white'
                          : 'bg-gray-100 text-gray-700'
                      }`}>
                        {option.id}
                      </span>
                      <span className="font-kalpurush text-gray-700">
                        {option.text}
                      </span>
                    </div>

                    {showResult && (
                      <div className="flex items-center">
                        {isCorrect ? (
                          <CheckCircle className="w-5 h-5 text-green-500" />
                        ) : (
                          <XCircle className="w-5 h-5 text-red-500" />
                        )}
                      </div>
                    )}
                  </div>
```

```
            </div>
          );
        })}
      </div>

      {/* Solution Toggle */}
      {question.solution && (
        <div className="pt-4">
          <Button
            variant="outline"
            onClick={() => toggleSolution(question.id)}
            className="flex items-center gap-2"
            data-testid={`button-solution-${question.id}`}
          >
            <Lightbulb className="w-4 h-4" />
            {showSolutions[question.id] ? 'Hide Solution' : 'Show Solution'}
          </Button>

          {/* Solution Box */}
          {showSolutions[question.id] && (
            <div className="mt-4 p-4 bg-green-50 border border-green-200 rounded-lg">
              <div className="flex items-center gap-2 mb-3">
                <Lightbulb className="w-5 h-5 text-green-600" />
                <h4 className="font-semibold text-green-800 font-kalpurush">সমাধান:</h4>
              </div>

              {/* Solution Images */}
              {question.solutionImages && question.solutionImages.length > 0 && (
                <div className="flex flex-wrap gap-2 mb-3">
                  {question.solutionImages.map((imageUrl, index) => (
                    <div key={index} className="inline-block rounded overflow-hidden
shadow-sm bg-white">
                      <img
                        src={imageUrl}
                        alt={`Solution image ${index + 1}`}
                        className="max-w-xs max-h-40 object-contain"
                        onError={(e) => {
                          const target = e.target as HTMLImageElement;
                          target.style.display = 'none';
                        }}
                        data-testid={`img-solution-${question.id}-${index}`}
                      />
                    </div>
                  ))}
```

```
                  </div>
                )}

                <div className="text-gray-800 mathematical-solution font-kalpurush">
                  {question.solution}
                </div>
              </div>
            )}
          </div>
        )}
      </CardContent>
    </Card>
  ))}
</div>

{/* Summary */}
<div className="max-w-4xl mx-auto px-4 pb-8">
  <Card className="bg-white">
    <CardContent className="p-6">
      <div className="text-center">
        <h3 className="text-lg font-semibold text-gray-900 mb-4 font-kalpurush">
          প্রশ্ন সমাপ্ত
        </h3>
        <div className="grid grid-cols-3 gap-4 mb-4">
          <div>
            <p className="text-2xl font-bold
text-blue-600">{Object.keys(firstClickAnswers).length}</p>
            <p className="text-sm text-gray-600">Answered</p>
          </div>
          <div>
            <p className="text-2xl font-bold text-green-600">
              {Object.entries(firstClickAnswers).filter(([questionId, optionId]) => {
                const question = quizData.questions.find(q => q.id === parseInt(questionId));
                return question && isCorrectAnswer(question, optionId);
              }).length}
            </p>
            <p className="text-sm text-gray-600">Correct</p>
          </div>
          <div>
            <p className="text-2xl font-bold text-purple-600">{calculateAccuracy()}%</p>
            <p className="text-sm text-gray-600">Accuracy</p>
          </div>
        </div>
        <p className="text-gray-600 mb-4">
```

```
              {Object.keys(firstClickAnswers).length} out of {quizData.questions.length} questions
answered
            </p>
            <Button
              onClick={() => setLocation('/')}
              className="mt-4"
              data-testid="button-upload-another"
            >
              Upload Another File
            </Button>
          </div>
        </CardContent>
      </Card>
    </div>
  </div>
 );
}
```